Jyotsna Bellary

Exam: Efficient Route Planning

1 Part 1

1.1

Table 1: CH results for each dataset

File	Runtime (ms)	Shortcuts
osm1	58	381
osm2	106	816
osm3	305	1764
osm4	709	4395
osm5	1380	9035
osm6	2653	18346
osm7	6965	46100
osm8	15575	93605
osm9	34139	189198
osm10	94271	474703
osm11	202524	940156

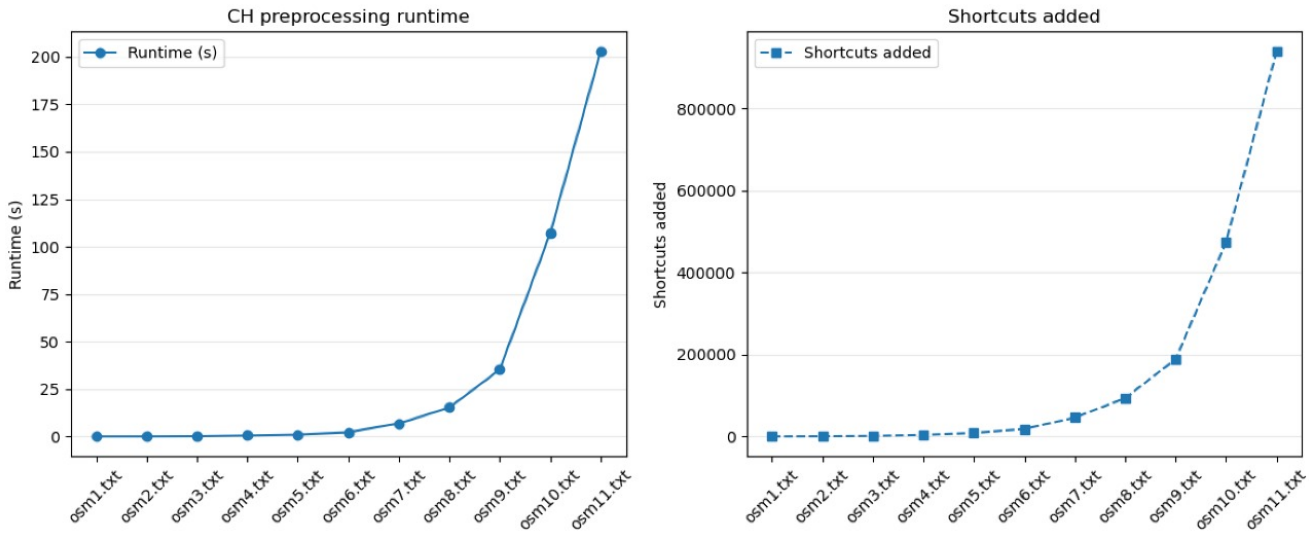

Figure 1: CH Results Plotted

Note: The Processing time for CH took about 4 minutes for the largest graph. Independent nodes were contracted and any node with Edge Difference less than the Median of all active nodes was contracted. This helped improve the pre-processing time.

1.2

Table 2: Average running times, priority queue operations, and identical counts across datasets

Dataset	Dijkstra		CH		Identical
	Avg. Time (ms)	PQ pops	Avg. Time (ms)	PQ pops	Count
osm1	0.20	265.19	0.00	24.01	100
osm2	0.68	510.92	0.00	29.69	100
osm3	2.02	942.05	0.01	37.22	100
osm4	6.75	2704.37	0.01	48.62	100
osm5	14.90	5076.34	0.05	52.76	100
osm6	32.98	10068.24	0.43	71.19	100
osm7	98.14	26427.31	2.37	92.62	100
osm8	220.49	54251.33	4.23	126.71	100
osm9	430.81	100893.26	7.21	177.76	100
osm10	1296.19	268571.72	20.31	285.17	100
osm11	2706.06	539961.73	35.99	347.37	100

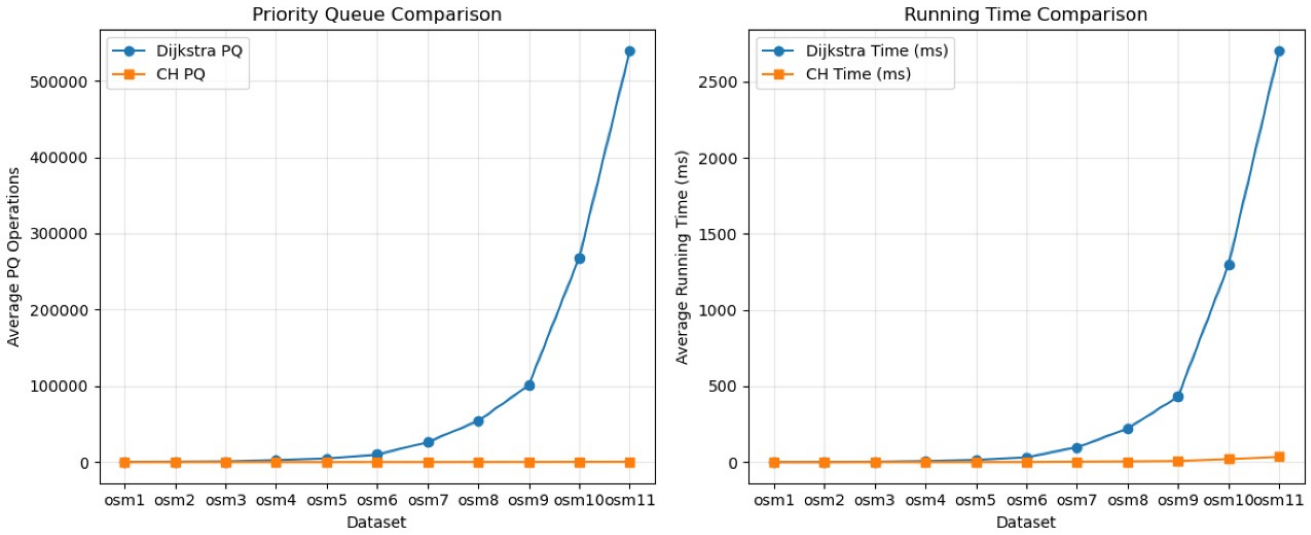

Figure 2: a) Dijkstra and CH Query Times over 100 runs. b) Average number of pops over 100 runs.

1.3

The pictures of four different paths can be found in the Appendix of this Report. The pictures with the Dijkstra and CH Dijkstra path have been placed next to each other for comparison. The paths seem to be very similar with no major shifts in the directions of the path.

2 Part 2

2.1

Table 3: Separator decomposition and pre-processing results

File	Separator Decomp	Preprocessing	Shortcuts	△ (Avg.)	△ (Max.)
osm1	16	31	338	1	7
osm2	114	83	771	2	22
osm3	263	152	1759	2	28
osm4	696	1028	4317	2	45
osm5	1289	3229	8946	2	49
osm6	2584	12062	18129	2	61
osm7	9044	80148	46691	2	76
osm8	17213	312821	94218	2	84
osm9	37005	1409193	197603	2	110
osm10	106525	7945303	493808	2	174
osm10	-	-	915847	-	-

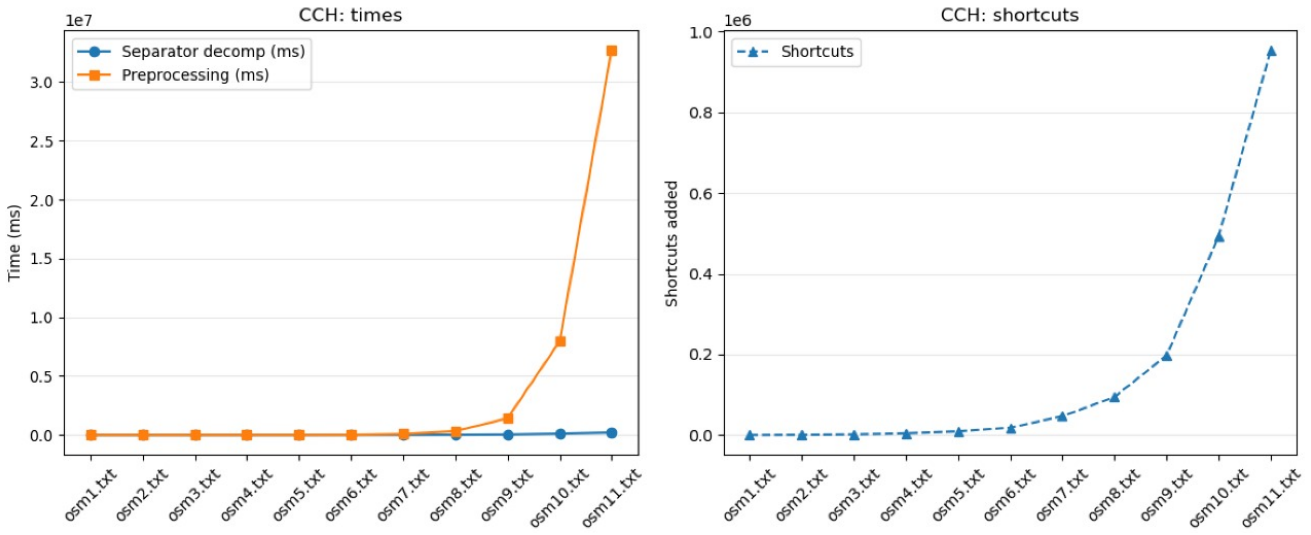

Figure 3: a) Seperator Decomposition and Processing time for CCH. b) Number of Graphs added

The node ordering function from KaHIP was used to obtain the separator decomposition. The result obtained was of the manner Node id and its rank in the contraction order. Unfortunately the separator and pre-processing time was lost but as observed it is worse than it is to be expected already for OSM9 and OSM10. The number of shortcuts however was available from the consolable and is hence reported. However, the customization time is fast. Overall the number of shortcuts seem comparable to the shortcuts added in the CH Pre-processing.

2.2

Table 4: Orig. and avg. of customization times

File	Original (ms)	Avg of 5 Runs (ms)
osm1	0	0
osm2	0	0.2
osm3	0	1.0
osm4	2	3.4
osm5	4	8.6
osm6	7	32.6
osm7	21	71.2
osm8	52	199.6
osm9	150	453.8
osm10	439	1200.4

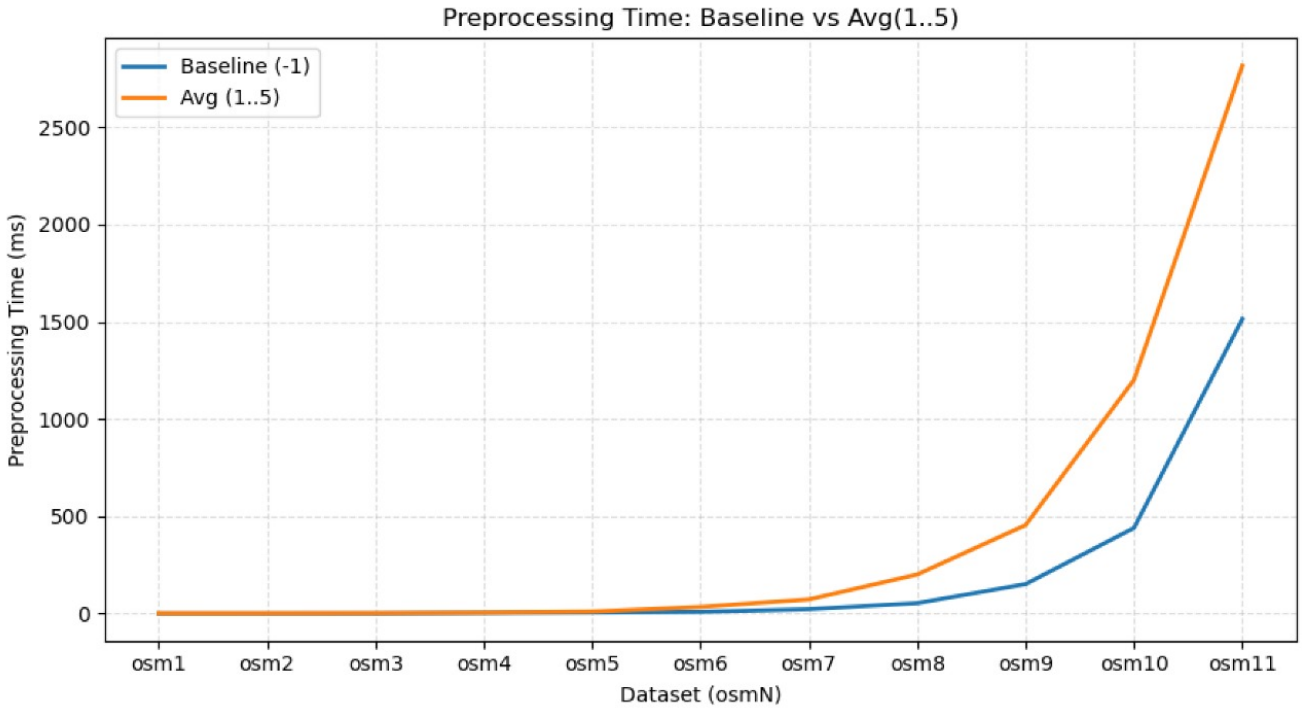

Figure 4: Original Customization time and Average of customization run 5 times with random graphs

2.3

Table 5: Average running times, priority queue operations, and identical counts across datasets

Dataset	Dijkstra		CH		CCH		Identical
	Avg. (ms)	PQ pops	Avg. (ms)	PQ pops	Avg. (ms)	PQ pops	Count
osm1	1.42	265.19	0.00	24.01	0.01	34.17	100
osm2	3.29	510.92	0.03	29.69	0.01	38.77	100
osm3	3.69	942.05	0.01	37.22	0.00	52.15	100
osm4	14.00	2704.37	0.00	48.62	0.06	65.92	100
osm5	29.51	5076.34	0.09	52.76	0.26	74.86	100
osm6	58.02	10068.24	0.23	71.19	0.37	104.06	100
osm7	150.48	26427.31	1.62	92.62	1.46	131.67	100
osm8	359.79	54251.33	3.25	126.71	3.53	169.12	100
osm9	805.52	100893.26	7.93	177.76	7.39	219.13	100
osm10	2030.57	268571.72	18.43	285.17	17.14	361.05	100

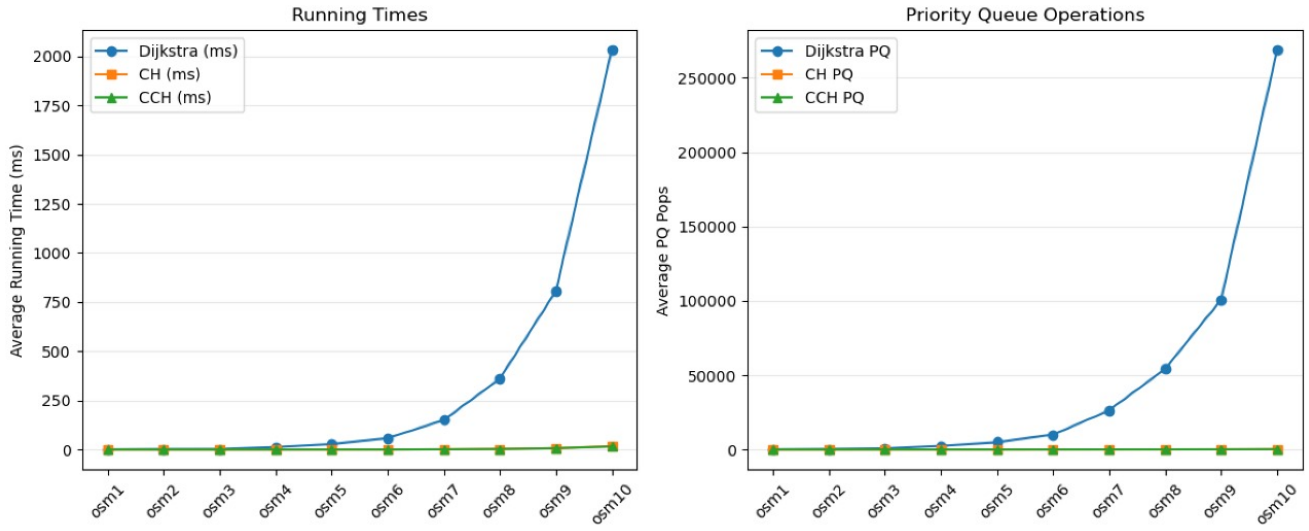

Figure 5: Dijkstra, CH and CCH Query Times over 100 runs. b) Average number of pops over 100 runs.

The number of Priority Queue pops are more compared to those of Dijkstra for all files. However, the CCH Query is slightly faster, if not the same for most of the Graphs. Both CH and CCH queries for shortest paths are observed to be much faster than Dijkstra queries.

3 Part 3

Application Description

The application is an interactive route planning web tool that allows users to upload a road network file, visualize it on a map, and query shortest paths using different algorithms. Specifically, the user can run:

- Dijkstra's algorithm

- Contraction Hierarchies (CH) Dijkstra
- Customizable Contraction Hierarchies (CCH) Dijkstra

The application supports displaying the shortcut paths created by CH/CCH alongside the fully expanded actual paths, allowing direct comparison of how hierarchical shortcuts accelerate queries.

The Web Client uses Leaflet.js for map rendering and visualization. The User needs to upload the Graph file. The user can choose the source and destination and choose the algorithm to use for querying the shortest path. The layers used to manage the visual elements are the Node Layer, Edge Layer, Route and Shortcut Layer. They can toggle wether to display the shortcuts. The web communicates with the Node.js backend over REST endpoints. The backend then spawns the C++ CLI executable and passes arguments necessary for the query. It reads, parses and send result back to the client to be drawn on the map. The default is set to OSM1. To use other graphs, the graph name needs to be updated in the code base.

A Appendix

All figures on the left are of the Dijkstra Query Result and the ones on the right are from the CH Query with the shortcuts. Not much of a difference were observed between the paths.

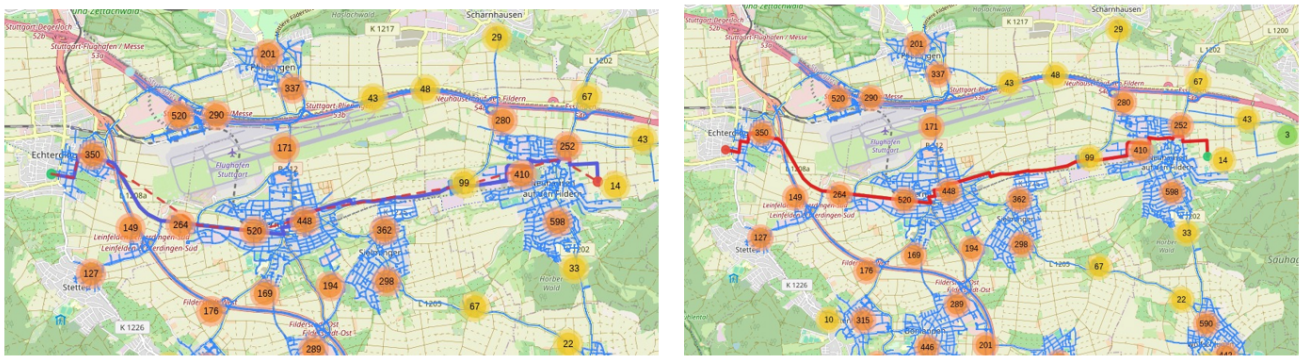

Figure 6: Snapshot 1

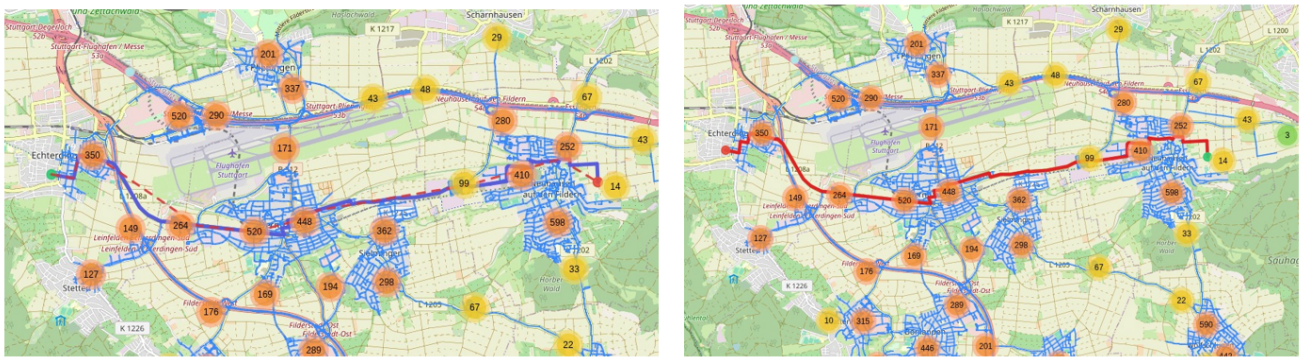

Figure 7: Snapshot 2

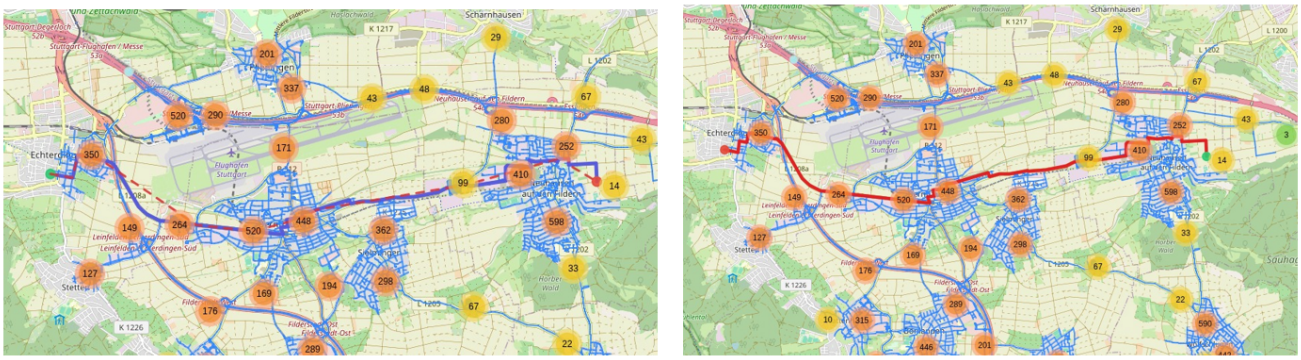

Figure 8: Snapshot 3

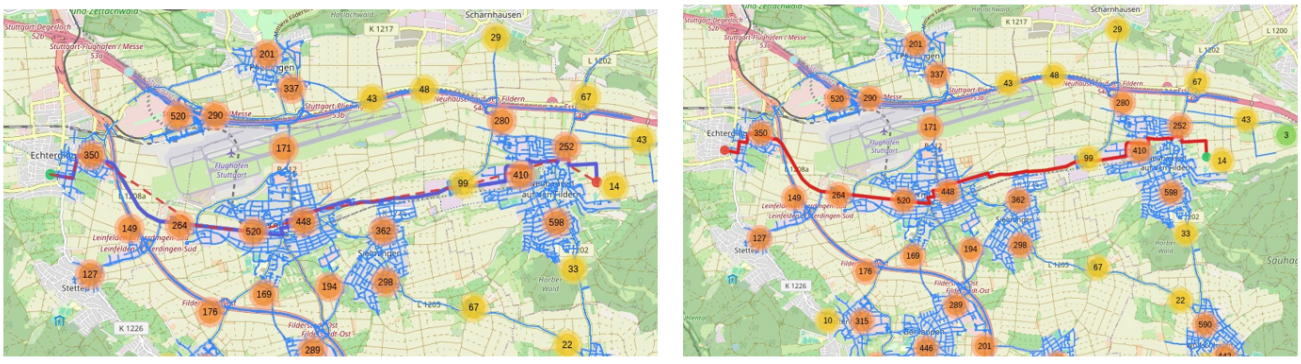

Figure 9: Snapshot 4.